

Lab: Public-Key Cryptography, PKI and TLS Certificates

In this lab, you will explore how public-key cryptography and Public Key Infrastructure (PKI) are used to provide confidentiality and establish trust. You will investigate the problem of trusting a public key and see how digital certificates and Certificate Authorities (CAs) address this problem. Finally, you will examine how the same concepts are used by a real HTTPS website.

Learning Objectives

By the end of this lab, you should be able to:

- Generate an RSA public/private key pair using Python.
- Use an RSA public key to encrypt a message.
- Use an RSA private key to decrypt a message.
- Inspect the certificate of a real HTTPS website.
- Understand the basic structure of Public Key Infrastructure (PKI).

Install the required Python library:

```
pip install cryptography
```

Create a directory for the lab:

```
pki_lab/
```

Exercise 1: Public-Key Encryption

Consider the scenario in which Alice wants to send Bob a secret message over an insecure network.

Bob uses a public/private key pair.

The important property of public-key cryptography is that Bob can make his public key available to Alice without revealing his private key.

Alice can then use Bob's public key to encrypt a message that only Bob can decrypt using his private key.

1.1. Use Python and the *cryptography* library to generate a 2048-bit RSA private key for Bob. Extract Bob's public key. Keep Bob's private key separate. Make Bob's public key available to Alice.

```
bob_private.pem
```

```
bob_public.pem
```

Create Alice's message for Bob.

Create a text file:

alice_message.txt

For example: "Hello Bob, the meeting is at 14:00. Alice"

1.2. Encrypt the message

Load Bob's public key and use it to encrypt Alice's message.

Save the encrypted data as:

encrypted_message.bin

1.3. Decrypt the message

Now simulate Bob receiving the ciphertext.

Load Bob's private key. Decrypt encrypted_message.bin.

Save the result as:

decrypted_message.txt

Compare the original and decrypted files. Are messages identical?

You have shown that PKC provides confidentiality without Alice and Bob first sharing a secret key. Public-key encryption introduces another problem.

Suppose Alice receives a public key claiming to belong to Bob. But how does Alice know that this public key she received actually belongs to Bob?

Exercise 2. The Public-Key Authentication Problem

Simulate an attack in which Alice thinks she has received Bob's public key. An attacker (Eve) replaces Bob's public key with her own public key. Alice does not know that the key has been replaced, so she encrypts her message using the substituted public key. Eve receives the ciphertext. Eve decrypts it using her private key and she can recover Alice's message.

Note! RSA encryption alone does not establish the identity associated with a public key.

This is why we need digital certificates.

Digital Certificates and Certificate Authorities

Instead of simply giving Alice a public key, Bob can provide a **digital certificate**.

A certificate can be thought of as a signed statement containing information such as Identity, Public key: Bob's public key, Validity period, Issuer, CA and CA's signature.

The CA's signature allows others (for example, Alice) to verify that the certificate was issued by that CA.

The certificate therefore provides a way to associate an **identity with a public key**, while the CA provides a mechanism for establishing trust in that association.

2.1. Investigate a Real HTTPS Certificate

You will now investigate how this works in a real HTTPS connection.

Choose a HTTPS website. For example: *https://www.google.com*

Use your browser's security certificate information to inspect the website TLS certificate.

Record the following:

Website/domain

Subject

Issuer

Valid from

Valid until

Public-key algorithm

Signature algorithm

Subject Alternative Names (SANs)

2.2. Certificate chain

Find the certificate chain shown by your browser.

Represent it as:

Root Certificate Authority

↓

Intermediate Certificate Authority

↓

Website Certificate

Record the actual names of the certificates shown by your browser.

2.3. Certificate validation

Determine:

1. Is the certificate currently valid?
2. Which domains does the certificate cover?

3. Who issued the certificate?
4. Which root CA ultimately establishes trust?
5. Are there intermediate certificates?
6. What public-key algorithm is used by the website certificate?